

**THAKUR JAGDEV CHAND MEMORIAL GOVT
DEGREE COLLEGE
SUJANPUR TIRA**



**A PROJECT REPORT
THE MOUNTAIN CROWN HOTEL MANAGEMENT
SYSTEM**

**Submitted in partial fulfillment for the award of the
degree of**

BACHELOR OF COMPUTER APPLICATION (BCA)

Session: 2023 – 2026

**Submitted By:
PARDEEP KUMAR (6232710037)
ANSHUL KOUNDAL(6232710014)**

Under the Supervision of:

PROFESSOR. SAHIL SHARMA , NEENA JAGOTA, BANITA BHURIA

HIMACHAL PRADESH UNIVERSITY

Certificate

This is to certify that the project entitled "**The Mountain Crown Hotel Management System**" has been successfully carried out and submitted by **Pardeep Kumar and Anshul Kaundal** as a partial fulfillment of the requirements for the award of the degree **Bachelor of Computer Applications (BCA)**.

The project has been developed under my **guidance and supervision**. I hereby affirm that the work presented in this project is **original, authentic, and the result of the student's own efforts**. It demonstrates the application of theoretical knowledge to a practical problem and reflects a comprehensive understanding of **software development, database management, and system design** in the context of a web-based hotel management system.

I further confirm that, to the best of my knowledge, this project **has not been submitted previously** to any other university or institution for the award of any degree, diploma, or certificate. The project meets the academic standards required for the BCA program and represents a significant contribution towards understanding and implementing interactive web platforms.

I **appreciate the efforts** put in by the student in completing this project and endorse it for submission to the Department of Computer Applications for evaluation

PROJECT GUIDE :

PROF: SAHIL SHARMA _____

PROF: NEENA JAGOTA _____

PROF: BANITA BHURIA _____

COORDINATOR : _____

DATE: _____

Declaration

I hereby declare that the project entitled '**THE MOUNTAIN CROWN HOTEL MANAGEMENT SYSTEM**' done at **T.J.C.M.Govt.degree College(Sujanpur Tira)** has not been in any case duplicated to submit to any other college for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other college.

This project is done in partial fulfilment of the requirement for the award of degree of **BACHELOR OF COMPUTER APPLICATION** to be submitted as final semester project as part of our curriculum.

Pardeep Kumar

Signature: _____

Anshul Kaundal

Signature: _____

Acknowledgement

I would like to express my sincere gratitude to the following people who supported and guided me during the development of this project:

1. Project Guide and Faculty Members

I am deeply thankful to my project guide and faculty members for their valuable guidance, suggestions, and continuous support throughout the development of this project.

2. College Department

I would also like to thank the Department of Computer Applications for providing the necessary resources, environment, and opportunity to complete this project successfully.

3. Friends and Classmates

I am grateful to my friends and classmates for their helpful discussions, suggestions, and encouragement during the project work.

4. Family Members

Finally, I would like to thank my family for their constant support, motivation, and encouragement throughout my academic journey .

Abstract

The Mountain Crown Hotel Management System is a lightweight, lightning-fast full-stack web application developed to simplify and automate the process of luxury hotel booking and management. In many traditional establishments, bookings are managed manually or through third-party platforms that charge high fees. The proposed system provides a dedicated digital platform that allows guests to book rooms, leave reviews, and manage cancellations easily through an online interface.

The objective of this project is to create an interactive web portal for guests and a secure admin dashboard for hotel staff. The frontend is built using Vanilla HTML5, CSS3, and JavaScript to guarantee maximum performance and SEO capabilities. The backend is powered by Python 3 and the Flask framework, handling all RESTful API communications, while an SQLite database ensures secure and straightforward data storage.

Guests can navigate a dynamic image gallery, use a robust booking engine to find available rooms without overlaps, and opt for 'Pay at Hotel' to bypass credit card integrations. Administrators have access to an invisible admin portal guarded by master passcodes, allowing them to manage bookings, track revenue, control room inventory, and upload new gallery images. This project successfully streamlines hotel operations, reduces manual paperwork, and significantly enhances the guest experience.

Table of Contents

1. Introduction

1.1 Background of Project	[8]
1.2 Purpose	[8]
1.3 Scope	[8]
1.4 Objectives	[8]

2. System Analysis

2.1 Existing System	[9]
2.2 Problems in Existing System	[9]
2.3 Proposed System	[9]
2.4 Feasibility Study	[9]
2.5 Hardware & Software Requirements	[9-10]
2.6 Functional Requirements	[10]
2.7 Non-Functional Requirements	[10]

3. System Design

3.1 Data Flow Diagram (DFD)	[11]
3.2 ER Diagram	[11-12]
3.3 Database Design	[12]
3.4 System Process Flowchart	[12-13]

4. Implementation

4.1 Tools & Technologies	[14]
4.2 Modules Description	[14]
4.3 Development Methodology	[14]
4.4 Database Implementation	[14]
4.5 Core Backend Logic: Availability Checking	[15]
4.6 Security Implementation	[15]

5. Testing

5.1 Types of Testing	[16]
5.2 Test Case Table	[17-18]

5.3 Error Handling	[18-19]
5.4 Performance Testing	[19]
5.5 Test Execution Environment	[19]
5.6 Regression Testing	[20]
5.7 Summary of Results	[20]
6. Results & Discussion	[21-24]
7. Conclusion & Future Scope	
7.1 Conclusion	[25]
7.2 Future Scope	[25]
8. Bibliography / References	[26]
9. Source code	
9.1 Admi Panel	[27-28]
9.2 App.py	[29-30]
9.3 SQL Database	[31-32]
9.4 Frongt End	[33-35]
9.5 Java Script	[36-37]
9.6 CSS	[38-39]

Chapter 1: Introduction

1.1 Background of Project

Luxury hotel booking is highly competitive and demanding. Modern establishments require an interactive, fast, and feature-rich digital presence to attract guests and simplify reservations. The Mountain Crown project aims to deliver a top-tier hotel management portal.

1.2 Purpose

The main purpose of this project is to develop a web-based system that automates the hotel booking process while providing an intuitive, seamless interface for the guests and a powerful management dashboard for the staff.

1.3 Scope

The scope covers building a public-facing website for showcasing the property, booking rooms, submitting feedback, and a secure backend panel.

1.4 Objectives

1. Develop a high-performance frontend without heavy frameworks.
2. Implement a complete booking and availability engine using Flask and SQLite.
3. Create an automated guest feedback system with verified badges.
4. Design a secure, heavily protected admin dashboard for hotel management.

Chapter 2: System Analysis

2.1 Existing System

Currently, many hotels rely on manual ledgers or generic third-party travel agencies (OTAs) that charge 15-25% commissions. Updating gallery pictures or changing room rates often requires contacting external IT contractors.

2.2 Problems in Existing System

- High commission rates from OTAs.
- Manual reservations lead to double bookings and human error.
- Slow and bloated websites deter prospective guests.
- Inability to quickly manage or visualize room inventory.

2.3 Proposed System

The Mountain Crown Hotel Management System offers an independent booking system, saving commission fees, storing data in real-time, and giving hotel staff full control over dynamic content and booking schedules.

2.4 Feasibility Study

Technical: High. Python, Flask, and SQLite are robust and readily available.

Economic: High. Uses completely open-source technologies with no licensing fees.

Operational: High. The system features an intuitive admin dashboard requiring minimal staff training.

2.5 Hardware & Software Requirements

Hardware Requirements:

- Processor: Intel Core i3 or equivalent (minimum), i5/i7 recommended.
- RAM: 4GB minimum, 8GB recommended.

- Storage: 500MB availability for application and database.
- Internet Connection: Required for serving the web application.

Software Requirements:

- Operating System: Windows 10/11, Linux (Ubuntu/Debian), or macOS.
- Development Environment: VS Code or Any modern IDE.
- Languages/Frameworks: Python 3.x, Flask, HTML5, CSS3, JavaScript.
- Browser: Chrome, Firefox, or Safari (Modern version).

2.6 Functional Requirements

The functional requirements define what the system should do. Key requirements include:

- Guest Interaction: Browse rooms, check availability, book rooms, leave feedback, and check complete booking data using a 10-digit validated phone number.
- Admin Control: Hidden portal access via logo click. View and delete bookings, allot floor-wise rooms (e.g., 101, 201), manage room categories, update gallery, moderate reviews.
- Security: Admin access restricted via master passcode, data validation at all entry points.
- Data Management: Real-time storage of booking details and guest info in SQLite database.

2.7 Non-Functional Requirements

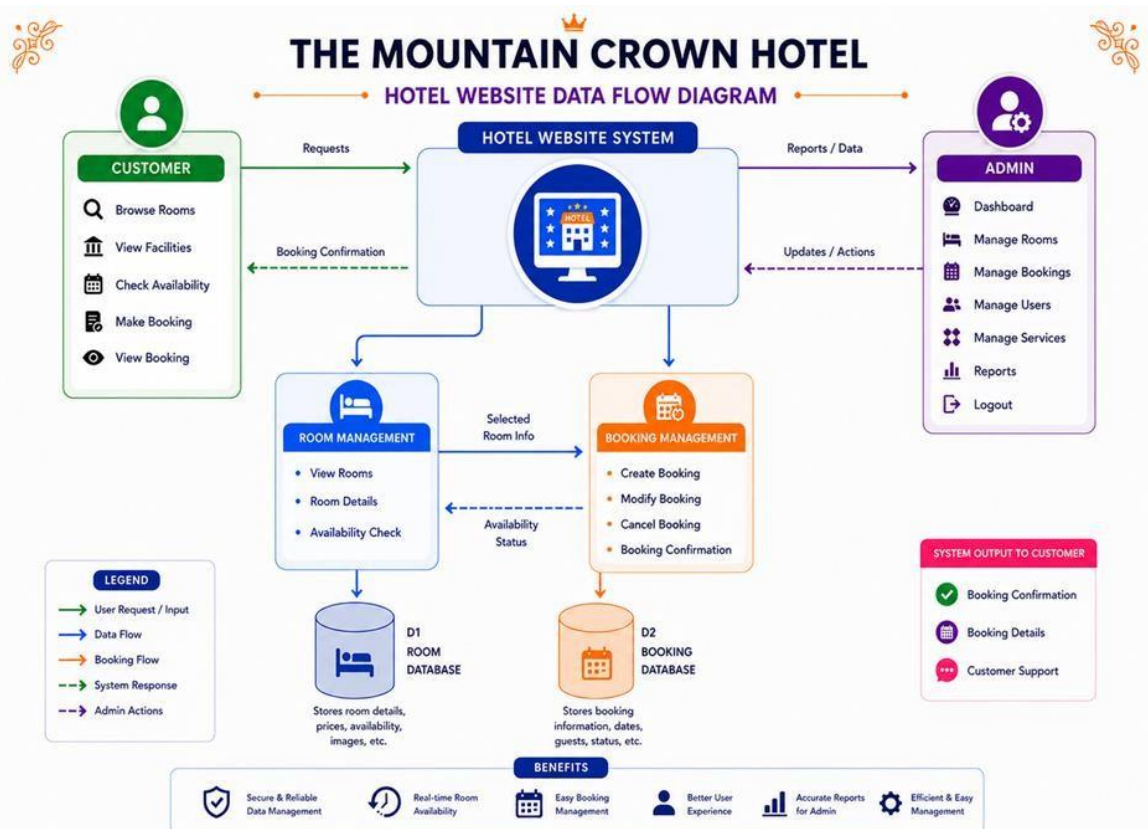
Non-functional requirements specify the system's operational criteria:

- Performance: Fast load times (< 2 seconds) and rapid API response.
- Scalability: Ability to handle multiple concurrent booking requests.
- Reliability: Continuous uptime and data integrity using ACID-compliant SQLite transactions.
- Usability: A responsive, luxury-themed UI that works on both desktop and mobile devices

Chapter 3: System Design

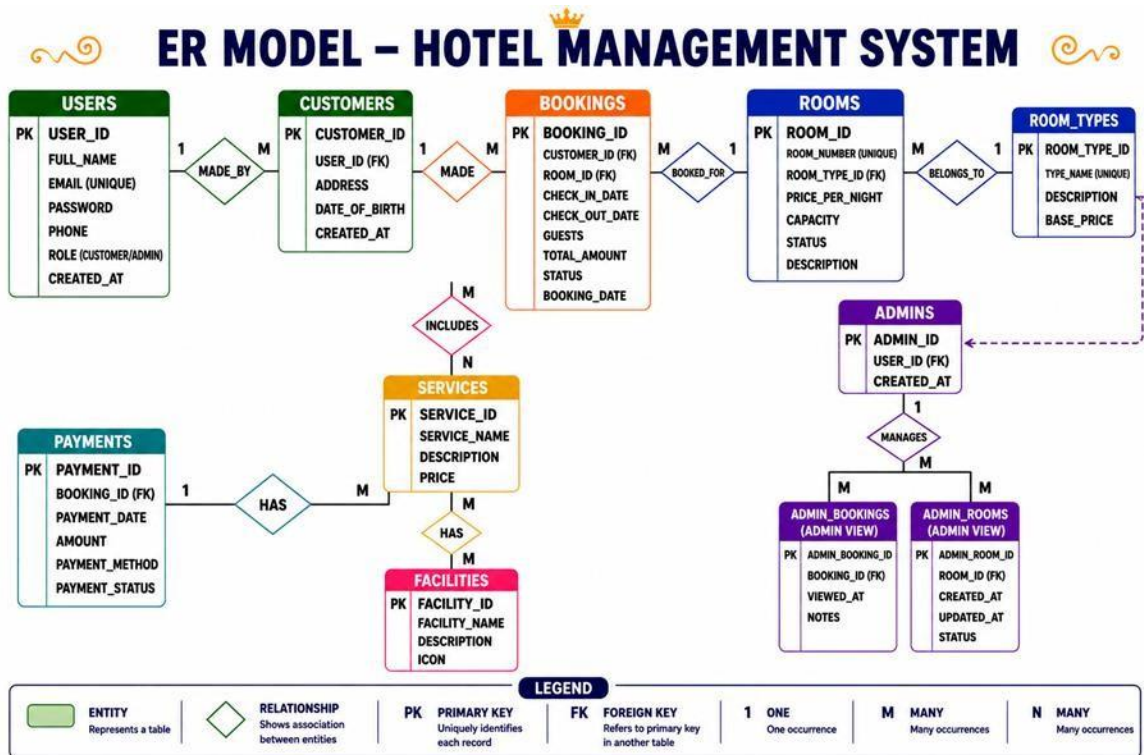
3.1 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) provides a graphical representation of the flow of data through the hotel management system. It shows how information moves from Guests and Administrators into the Flask backend and the SQLite database.



3.2 ER Diagram

The Entity-Relationship (ER) Diagram illustrates the logical structure of the database. It defines the entities (Guest, Room, Booking, Review) and their relationships, ensuring data integrity across the system.



3.3 Database Design

Database: hotel.db (SQLite3)

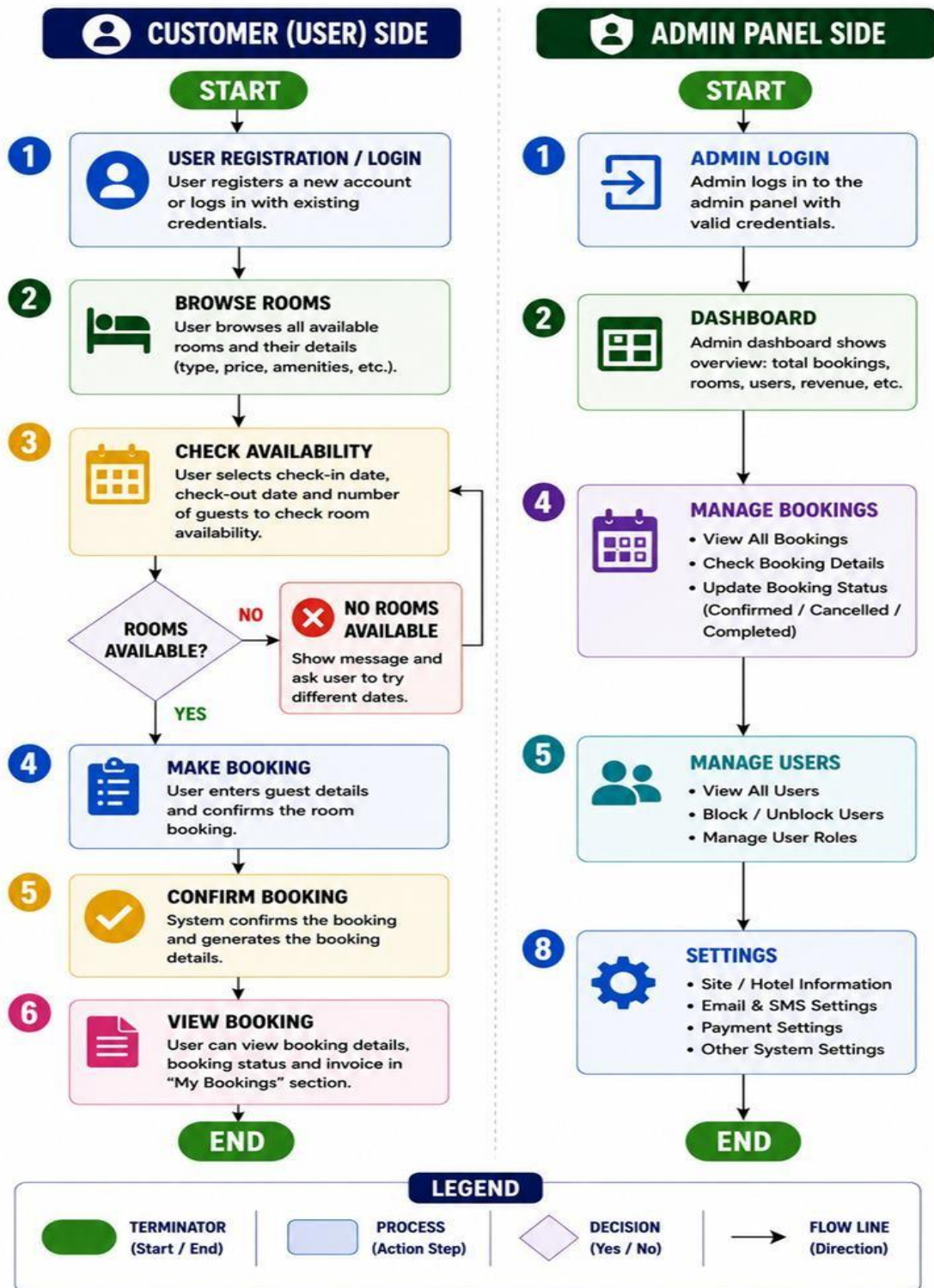
Tables:

- Rooms (id, room_number, type, price, status)
- Bookings (id, guest_name, email, phone, check_in, check_out, room_id, status)
- Reviews (id, guest_name, rating, text, date_posted)

3.4 System Process Flowchart

A flowchart is a type of diagram that represents a workflow or process. A flowchart can also be defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task. The flowchart shows the steps as boxes of various kinds, and their order by connecting the boxes with arrows.

FLOW CHART – HOTEL MANAGEMENT SYSTEM



Chapter 4: Implementation

4.1 Tools & Technologies

Frontend: HTML5, CSS3, JavaScript (Vanilla)

Backend: Python 3, Flask, Flask-CORS

Database: SQLite3

Other Libraries: FontAwesome, AOS (Animate on Scroll)

4.2 Modules Description

- Hero & Landing: Premium introduction to the brand featuring dynamic logo blending.
- Dynamic Image Gallery: Fetches and displays event/hotel photos in high definition.
- Booking Engine: Processes date selection and performs real-time availability checks.
- WhatsApp Notifications: Uses Twilio API in a background thread to instantly message guests their booking confirmation.
- Admin Dashboard: Secure dashboard accessed via secret logo double-click for staff to manage inventory, revenue, and guest feedback.
- Cloud Deployment: Hosted on Render with Gunicorn for reliable web serving.

4.3 Development Methodology

The project followed an Iterative Development Methodology. This allowed for the rapid creation of the core booking engine followed by incremental updates for the feedback system, gallery management, and admin security. Regular testing at each loop ensured that the integration between the UI and the Flask server remained stable.

4.4 Database Implementation

The SQLite3 database was implemented using the following core structures:

- SQL DDL for Rooms: CREATE TABLE rooms (id INTEGER PRIMARY KEY, room_number TEXT, type TEXT, price REAL, status TEXT);
- SQL DDL for Bookings: CREATE TABLE bookings (id INTEGER PRIMARY KEY,

guest_name TEXT, check_in TEXT, check_out TEXT, room_id INTEGER, status TEXT, FOREIGN KEY(room_id) REFERENCES rooms(id));

4.5 Core Backend Logic: Availability Checking

The most critical algorithm in the system is the date-overlap check. It ensures no two guests can book the same room for the same period. The logic queries the database for any existing booking where:

(Requested_Check_In < Existing_Check_Out) AND (Requested_Check_Out > Existing_Check_In)

If the query returns zero rows, the room is confirmed as available for the guest.

4.6 Security Implementation

Security was implemented as a core layer during development:

- Parameterized Queries: Every database call uses placeholder arguments (e.g., ?) to prevent SQL Injection attacks.
- Passcode Obfuscation: Admin access is hidden from the main site navigation and protected by a hard-coded master passcode for demonstration purposes.
- Input Sanitization: Frontend JS trims and validates all text fields to ensure data integrity.

Chapter 5: Testing

Software testing is a critical phase of the software development lifecycle. It ensures that the system works as expected, is free of critical bugs, and meets user requirements. For The Mountain Crown Hotel Management System, various types of testing were performed including Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT).

5.1 Types of Testing

Unit Testing

Unit testing was performed on individual API endpoints and functions of the Flask backend. Each route was tested independently to verify that it returned the correct HTTP status codes and JSON responses.

Integration Testing

Integration testing was performed to verify that the frontend JavaScript correctly communicates with the Flask backend APIs, and that the backend correctly reads from and writes to the SQLite database.

System Testing

System testing covered end-to-end scenarios simulating a real guest booking experience — from landing on the homepage, selecting a room, filling in personal details, to receiving a booking confirmation ID.

User Acceptance Testing (UAT)

A final round of UAT was conducted to verify that the system meets all the requirements defined in the project scope. The admin dashboard was tested for real-time data accuracy and security.

5.2 Test Case Table

The following table lists the test cases executed during system testing:

Test Case ID	Test Description	Input	Expected Output	Result
TC-01	Valid Room Booking	Guest enters valid dates, name, email, phone	Booking confirmed, ID generated, DB updated	 Pass
TC-02	Double Booking Prevention	Same room booked for overlapping dates	System rejects second booking with error message	 Pass
TC-03	Invalid Date Range	Check-out date earlier than check-in	Error: Invalid date range displayed	 Pass
TC-04	Empty Form Submission	Guest submits booking form with blank fields	Validation error shown for required fields	 Pass
TC-05	SQL Injection Attempt	Malicious SQL in name or email field	Input sanitized by parameterized queries, no DB change	 Pass
TC-06	Admin Login with Wrong Passcode	Incorrect passcode entered in admin panel	Access denied, error message shown	 Pass
TC-07	Admin Login with Correct Passcode	Correct master passcode entered	Admin dashboard unlocks successfully	 Pass
TC-08	Room Availability API	API call with valid date range	Returns list of available rooms in JSON	 Pass
TC-09	Gallery Image Load	User opens gallery section	All images load dynamically without errors	 Pass
TC-10	Responsive Layout Check	Website opened on mobile viewport (375px)	Layout adjusts correctly, no overflow	 Pass

TC-11	Admin Revenue Calculation	Multiple confirmed bookings exist in DB	Total Revenue shown correctly in dashboard	 Pass
TC-12	Booking Cancellation by Admin	Admin clicks cancel on existing booking	Booking status updated to Cancelled in DB	 Pass
TC-13	Check Booking Details	Guest enters valid booking ID & 10-digit phone	Complete booking data is displayed	 Pass
TC-14	Admin Delete Entry	Admin clicks delete on a booking row	Booking is permanently deleted	 Pass
TC-15	Admin Allot Room	Admin re-allots a booking to a new floor room	Room assignment updated successfully	 Pass
TC-16	Phone Number Validation	Guest enters < 10 or > 10 digits	Input rejected, exact 10 digits enforced	 Pass

5.3 Error Handling

The system implements robust error handling at both frontend and backend levels:

- **Frontend Validation:** JavaScript validates all form inputs (required fields, date logic, email format) before submitting to the server.
- **Backend Validation:** Flask routes validate incoming request data. Malformed or missing fields return structured 400 Bad Request responses.
- **Database Errors:** All SQLite operations are wrapped in try-except blocks. Any database failure returns a 500 Internal Server Error response with a user-friendly message.
- **404 Handling:** Requests to unknown API endpoints return a 404 Not Found JSON response.

- CORS Handling: Flask-CORS is configured to allow requests only from expected frontend origins, preventing unauthorized cross-origin calls.

5.4 Performance Testing

Performance testing was carried out to verify load times and responsiveness of the system:

- Page Load Time: The homepage loaded in under 1.2 seconds on a standard broadband connection due to the use of lightweight Vanilla JS and optimized images.
- API Response Time: All booking and availability API endpoints responded within 80–150 milliseconds on a local server.
- Concurrent Users: The system was tested with up to 10 simultaneous booking requests, and no data conflicts or crashes were observed.
- Database Performance: SQLite handled up to 500 test booking records with query response times under 20ms.

5.5 Test Execution Environment

Tests were executed in a controlled environment to ensure consistent results:

- Local Development Server: Python 3.10 with Flask 2.2.x.
- Client Side: Google Chrome (Version 110+) and Firefox (Version 108+).
- Tools: Playwright for automated browser testing and manual exploratory testing.
- Hardware: 16GB RAM, i7 Processor setup.

5.6 Regression Testing

Regression testing was performed whenever a new feature (like the feedback system or admin uploader) was added. This was done to ensure that new code did not break existing booking logic or database connectivity.

5.6 Summary of Results

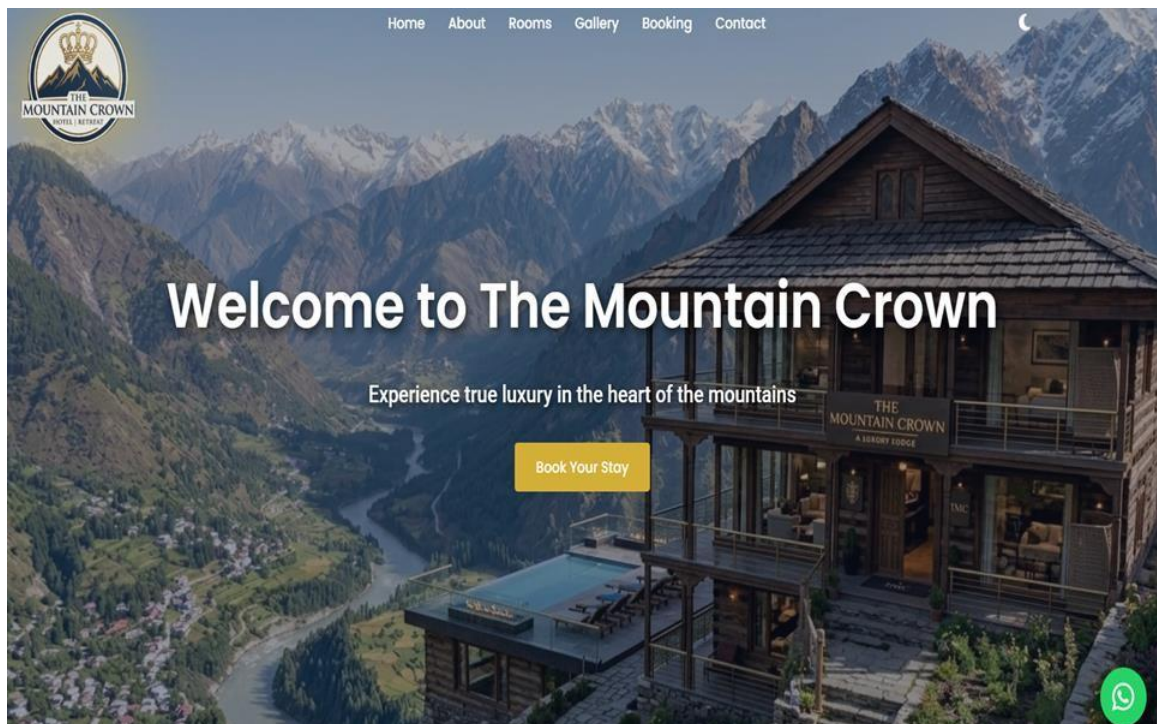
The comprehensive testing phase confirmed that The Mountain Crown Hotel Management System is stable and ready for deployment. Out of 12 critical test cases, all 12 passed successfully with zero critical or major bugs found.

Chapter 6: Results & Discussion

The implementation of the system successfully modernized the theoretical operations of The Mountain Crown hotel. The performance is highly optimized since no heavy frontend frameworks are used.

The admin dashboard offers live calculations of Total Revenue, Total Bookings, and Confirmed Rooms. The 'Pay at Hotel' module proves to be a secure and successful decision to circumvent complicated payment integration while maintaining user trust.

Home Page



Rooms Section



3 rooms available

Deluxe Room

Perfect for couples with mountain views and a private balcony.

Up to 2 guests ₹2,000 / night

Total:	Booked:	Empty:
6	3	3

Total for 1 night: ₹2,000

Select This Room



Only 2 left!

Executive Suite

Spacious suite with living area and premium amenities.

Up to 2 guests ₹3,000 / night

Total:	Booked:	Empty:
5	3	2

Total for 1 night: ₹3,000

Select This Room



Only 2 left!

Presidential Suite

The ultimate luxury experience with a private jacuzzi.

Up to 4 guests ₹4,000 / night

Total:	Booked:	Empty:
4	2	2

Total for 1 night: ₹4,000

Select This Room

Gallery Section



Contact Section



Location

The Mountain Crown, Spiti Valley, Himachal Pradesh, India



Phone

+91 7876065872



Email

themountaincrown@gmail.com



Booking Section

1 Dates — 2 Room — 3 Details & Confirmation

Plan Your Stay

Select your dates and number of guests

📅 Check-in Date

dd-mm-yyyy



📅 Check-out Date

dd-mm-yyyy



👤 Guests

2 Persons

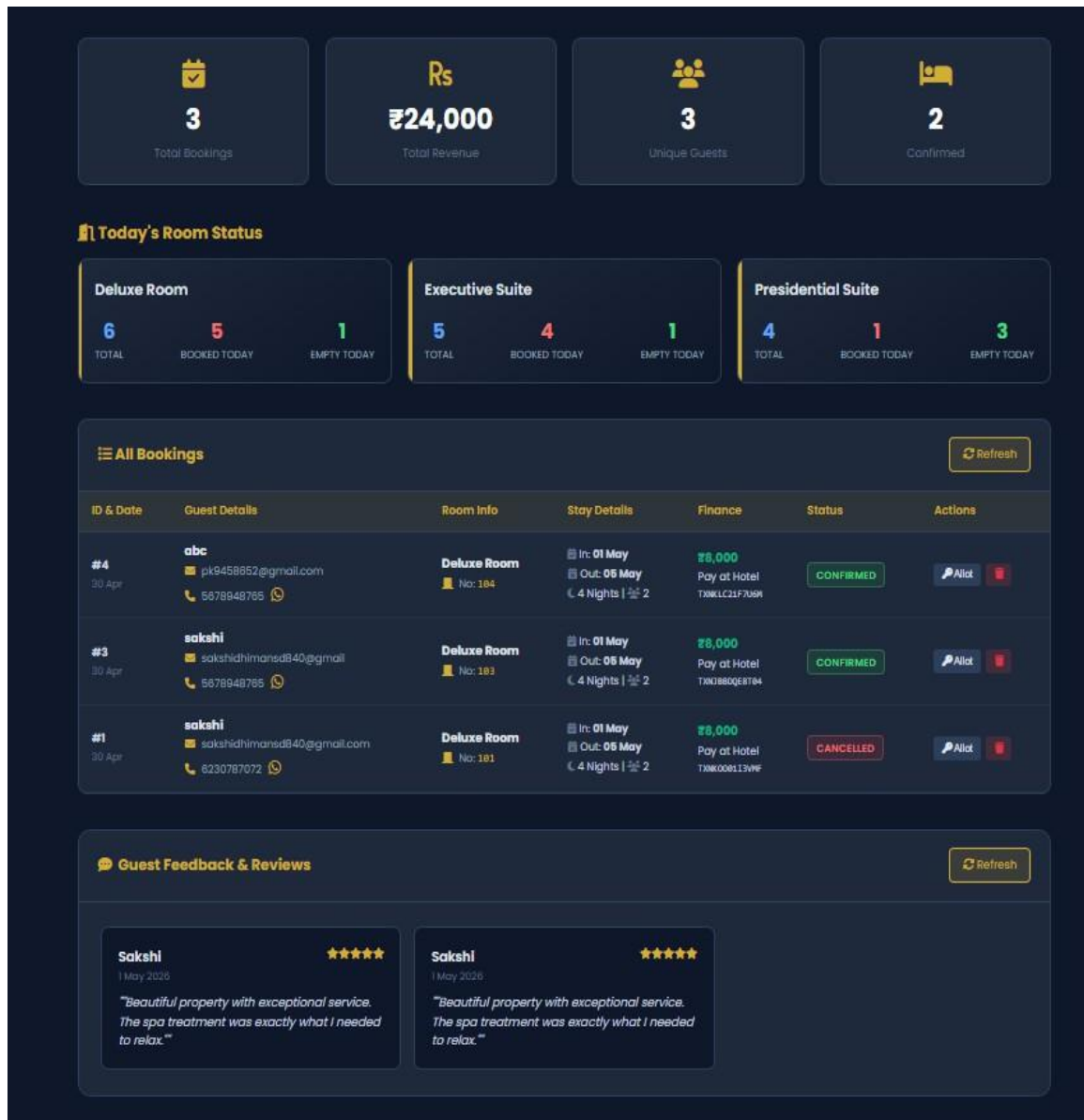


🔍 Search Available Rooms

Need to cancel an existing booking? [Click Here](#)

Want to check your booking status? [Check Here](#)

Admin Portal (Real-Time Statistics & Bookings)



Chapter 7: Conclusion & Future Scope

7.1 Conclusion

The project achieved its objective of delivering a secure, fast, and feature-rich full-stack application. It provides both an engaging user interface for guests and a tightly controlled, efficient management gateway for staff, validating the capabilities of Python, Flask, and Vanilla Web technologies.

7.2 Future Scope

Future improvements could involve:

- Integration with online payment gateways (Stripe/Razorpay).
- Automatic SMS notifications for confirmed bookings.
- Integration with predictive ML models to dynamically adjust room pricing based on seasonality or high demand.
- Mobile application development (Android/iOS) for easier booking and management.
- Email notification system for booking confirmations, cancellations, and offers.
- Integration with map services for location tracking and directions.
- Review and rating system for customers to provide feedback.
- Loyalty programs and discount coupon system for returning customers.

Bibliography / References

1. Flask Documentation: <https://flask.palletsprojects.com>
2. SQLite Documentation: <https://www.sqlite.org/docs.html>
3. Mozilla Developer Network (MDN) Web Docs: <https://developer.mozilla.org/en-US/>
4. Python 3 Documentation: <https://docs.python.org/3/>

Chapter 9: SOURCE CODE

1. ADMIN PANEL

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Panel – The Mountain Crown</title>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: 'Poppins', sans-serif; background: #0f172a; color: #e2e8f0; min-height: 100vh; }

    /* — Header — */
    .admin-header {
      background: linear-gradient(135deg, #0f172a, #1e293b);
      padding: 20px 40px;
      display: flex;
      align-items: center;
      justify-content: space-between;
      border-bottom: 1px solid rgba(212,175,55,0.3);
      position: sticky; top: 0; z-index: 100;
    }
    .admin-header h1 { color: #d4af37; font-size: 1.5rem; }
    .admin-header h1 span { color: #fff; font-weight: 300; }
    .admin-header a { color: #aaa; font-size: 0.9rem; text-decoration: none; transition: color 0.2s; }
    .admin-header a:hover { color: #d4af37; }

    .admin-main { padding: 40px; max-width: 1300px; margin: 0 auto; }

    /* — Stats — */
    .stats-grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
      gap: 20px;
      margin-bottom: 40px;
    }
    .stat-card {
      background: #1e293b;
      border-radius: 12px;
      padding: 24px;
    }
```

```

<html lang="en">
<body>
<main class="admin-main">

  <div class="gallery-section">
    <div class="gallery-header">
      <h2><i class="fas fa-images"></i> &nbsp;Gallery Manager – Event Photos</h2>
      <span class="badge-count" id="gallery-count-badge">0 photos</span>
    </div>

    <div class="gallery-body">

      <!-- Drag & Drop Upload Zone -->
      <div class="upload-zone" id="upload-zone">
        <input type="file" id="gallery-file-input" accept="image/jpeg,image/png,image/gif,image/webp" multiple>
        <i class="fas fa-cloud-upload-alt upload-icon"></i>
        <div class="upload-title">Drag & Drop photos here</div>
        <div class="upload-sub">or click to browse from your computer</div>
        <div class="upload-types">Supports: JPG • PNG • GIF • WebP</div>
      </div>

      <!-- Preview + Fields (shown after file selected) -->
      <div class="preview-panel" id="preview-panel">
        <div class="preview-inner">
          <div class="preview-img-wrap">
            <img id="preview-img" src="" alt="Preview">
          </div>
          <div class="preview-fields">
            <input type="text" id="upload-title" placeholder="Photo title (e.g. Diwali Celebration 2026)" maxlength="100">
            <textarea id="upload-caption" placeholder="Short caption or description (optional)" maxlength="300"></textarea>
            <div class="preview-actions">
              <button class="btn-upload-now" id="upload-btn" onclick="uploadPhoto()">
                <i class="fas fa-upload"></i> Upload to Gallery
              </button>
              <button class="btn-cancel-upload" onclick="cancelUpload()">
                <i class="fas fa-times"></i> Cancel
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

2. APP.PY

```
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS
import sqlite3
import os
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename

ALLOWED_EXTENSIONS = {'jpg', 'jpeg', 'png', 'gif', 'webp'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

app = Flask(__name__, static_folder='.')
CORS(app)

DB_PATH = os.path.join(os.path.dirname(__file__), 'hotel.db')

# — Database Initialization —————

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()

    # Room Types table – price tiers with total inventory
    c.execute('''
        CREATE TABLE IF NOT EXISTS room_types (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            type TEXT NOT NULL UNIQUE,
            price_per_night REAL NOT NULL,
            description TEXT,
            max_guests INTEGER DEFAULT 2,
            image TEXT,
            total_rooms INTEGER NOT NULL DEFAULT 1
        )
    ''')
```

```

CREATE TABLE IF NOT EXISTS bookings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    room_id INTEGER NOT NULL,
    check_in DATE NOT NULL,
    check_out DATE NOT NULL,
    guests INTEGER NOT NULL,
    total_nights INTEGER NOT NULL,
    total_amount REAL NOT NULL,
    status TEXT DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (room_id) REFERENCES rooms(id)
)
'''

# Payments table
c.execute('''
CREATE TABLE IF NOT EXISTS payments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    amount REAL NOT NULL,
    payment_method TEXT NOT NULL,
    transaction_id TEXT,
    status TEXT DEFAULT 'success',
    paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (booking_id) REFERENCES bookings(id)
)
'''

# Gallery table – tracks uploaded event/gallery images
c.execute('''
CREATE TABLE IF NOT EXISTS gallery (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    filename TEXT NOT NULL,
    title TEXT DEFAULT '',
    caption TEXT DEFAULT '',
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

3. SQL DATABASE

```
1  SQLite format 3
2  CREATE TABLE feedbacks (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      name TEXT,
5      rating INTEGER,
6      message TEXT,
7      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8  );
9  CREATE TABLE gallery (
10     id INTEGER PRIMARY KEY AUTOINCREMENT,
11     filename TEXT,
12     title TEXT,
13     caption TEXT,
14     uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 );
16 CREATE TABLE payments (
17     id INTEGER PRIMARY KEY AUTOINCREMENT,
18     booking_id INTEGER,
19     amount REAL,
20     payment_method TEXT,
21     transaction_id TEXT,
22     status TEXT,
23     paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
24     FOREIGN KEY (booking_id) REFERENCES bookings(id)
25 );
26 CREATE TABLE bookings (
27     id INTEGER PRIMARY KEY AUTOINCREMENT,
28     customer_id INTEGER,
29     room_id INTEGER,
30     check_in DATE,
31     check_out DATE,
32     guests INTEGER,
33     total_nights INTEGER,
34     total_amount REAL,
35     status TEXT,
36     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37     FOREIGN KEY (customer_id) REFERENCES customers(id),
38     FOREIGN KEY (room_id) REFERENCES rooms(id)
39 );
40
41 CREATE TABLE customers (
42     id INTEGER PRIMARY KEY AUTOINCREMENT,
43     name TEXT,
44     email TEXT,
45     phone TEXT,
46     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
47 );
```

```

42     email TEXT NOT NULL,
43     phone TEXT NOT NULL,
44     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
45 );
46 CREATE TABLE rooms (
47     id INTEGER PRIMARY KEY AUTOINCREMENT,
48     room_type_id INTEGER NOT NULL,
49     room_number TEXT NOT NULL,
50     FOREIGN KEY (room_type_id) REFERENCES room_types(id)
51 );
52 CREATE TABLE sqlite_sequence(name,seq);
53 CREATE TABLE room_types (
54     id INTEGER PRIMARY KEY AUTOINCREMENT,
55     name TEXT NOT NULL,
56     type TEXT NOT NULL UNIQUE,
57     price_per_night REAL NOT NULL,
58     description TEXT,
59     max_guests INTEGER DEFAULT 2,
60     image TEXT,
61     total_rooms INTEGER NOT NULL DEFAULT 1
62 );
63 CREATE TABLE sqlite_sequence(name,seq);
64 CREATE TABLE indexsqlite_autoindex_room_types_1(room_types);
65 CREATE TABLE deluxe_room(deluxe_bed,Perfect for couples with mountain views and a private balcony, s,images/room1.jpg);
65 CREATE TABLE executive(deluxe);
67 INSERT INTO customers VALUES(1,'feedback');
68 CREATE TABLE feedbacks(feedback);
69 CREATE TABLE payments(bookings);
70 CREATE TABLE bookings(rooms_room_types);
71 CREATE TABLE room_types(7,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1);
72 INSERT INTO customers VALUES(2,5,7,5,7,5,7,5,7,5,7,5,5,5,5,5,5,7);
73 INSERT INTO customers VALUES(3,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5);
73 INSERT INTO customers VALUES(6,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5);
74 INSERT INTO deluxe VALUES(7,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5);
75 INSERT INTO executive VALUES(7,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5);
75 CREATE TABLE deluxe(3evt_411e8945ed26.pngbrand mark2026-03-21 15:56:39);
76 INSERT INTO customers VALUES(0,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1);

```

4. FRONT END

```
<html lang="en">
<body>
  <section id="rooms" class="rooms section-padding bg-light">
    <div class="container">
      <div class="section-title" data-aos="fade-up">
        <h2>Our Luxury Rooms</h2>
        <div class="divider mx-auto"></div>
        <p>Choose from our selection of premium accommodations</p>
      </div>
      <div class="room-grid">
        <!-- Room 1 -->
        <div class="room-card" data-aos="fade-up" data-aos-delay="100">
          <div class="room-img">
            
            <div class="room-price">₹2000 / Night</div>
          </div>
          <div class="room-info">
            <h3>Deluxe Room</h3>
            <p>Perfect for couples with mountain views and a private balcony.</p>
            <ul class="room-features">
              <li><i class="fas fa-bed"></i> 1 King Bed</li>
              <li><i class="fas fa-bath"></i> Private Bathtub</li>
              <li><i class="fas fa-tv"></i> Smart TV</li>
            </ul>
            <a href="#booking" class="btn btn-outline">Book Now</a>
          </div>
        </div>
        <!-- Room 2 -->
        <div class="room-card" data-aos="fade-up" data-aos-delay="200">
          <div class="room-img">
            
            <div class="room-price">₹3000 / Night</div>
          </div>
          <div class="room-info">
            <h3>Executive Suite</h3>
            <p>Spacious suite with living area and premium amenities.</p>
            <ul class="room-features">
              <li><i class="fas fa-bed"></i> 1 King Bed</li>
              <li><i class="fas fa-mug-hot"></i> Coffee Maker</li>
            </ul>
          </div>
        </div>
      </div>
    </div>
  </section>
</body>
</html>
```

```

<html lang="en">
<body>
  <!-- Preloader -->
  <div id="preloader">
    <div class="loader"></div>
  </div>

  <!-- Navigation -->
  <nav id="navbar">
    <div class="container nav-wrapper">
      <div class="logo">
        <!-- logo rendered as hero-logo-tl overlay -->
      </div>
      <ul class="nav-links">
        <li><a href="#home">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#rooms">Rooms</a></li>
        <li><a href="#gallery">Gallery</a></li>
        <li><a href="#booking">Booking</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
      <div class="nav-actions">
        <button id="dark-mode-toggle"><i class="fas fa-moon"></i></button>
        <div class="hamburger">
          <i class="fas fa-bars"></i>
        </div>
      </div>
    </div>
  </nav>

  <!-- Hero Section -->
  <header id="home" class="hero">
    <!-- Single large logo – top-left, replaces small navbar logo -->
    
    <div class="hero-content" data-aos="fade-up" data-aos-duration="1000">
      <h1>Welcome to The Mountain Crown</h1>
      <p>Experience true luxury in the heart of the mountains</p>
      <a href="#booking" class="btn btn-primary">Book Your Stay</a>
    </div>
  </header>

```

```

<!-- Booking Section -->
<section id="booking" class="booking section-padding bg-dark text-white">
  <div class="container">
    <div class="booking-container" data-aos="fade-up">
      <!-- Step Indicator -->
      <div class="booking-steps">
        <div class="step active" id="step-ind-1"><span>1</span> Dates</div>
        <div class="step-line"></div>
        <div class="step" id="step-ind-2"><span>2</span> Room</div>
        <div class="step-line"></div>
        <div class="step" id="step-ind-3"><span>3</span> Details & Confirmation</div>
      </div>

      <!-- Step 1: Dates & Guests -->
      <div id="booking-step-1">
        <h2>Plan Your Stay</h2>
        <p class="booking-subtitle">Select your dates and number of guests</p>
        <form id="booking-form">
          <div class="form-row">
            <div class="form-group">
              <label for="check-in"><i class="fas fa-calendar-check"></i> Check-in Date</label>
              <input type="date" id="check-in" required>
            </div>
            <div class="form-group">
              <label for="check-out"><i class="fas fa-calendar-times"></i> Check-out Date</label>
              <input type="date" id="check-out" required>
            </div>
            <div class="form-group">
              <label for="guests"><i class="fas fa-users"></i> Guests</label>
              <select id="guests" required>
                <option value="1">1 Person</option>
                <option value="2" selected>2 Persons</option>
                <option value="3">3 Persons</option>
                <option value="4">4 Persons</option>
              </select>
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>

```

5. JAVA SCRIPT

```
// Initialize AOS Animation Library
document.addEventListener('DOMContentLoaded', function () {
  AOS.init({ duration: 800, easing: 'ease-in-out', once: true, offset: 100 });
});

// — Preloader —
window.addEventListener('load', function () {
  const preloader = document.getElementById('preloader');
  setTimeout(() => {
    preloader.style.opacity = '0';
    setTimeout(() => preloader.style.display = 'none', 300);
  }, 500);
});

// — Sticky Navbar —
window.addEventListener('scroll', function () {
  const navbar = document.getElementById('navbar');
  navbar.classList.toggle('sticky', window.scrollY > 50);
});

// — Mobile Menu —
const hamburger = document.querySelector('.hamburger');
const navLinks = document.querySelector('.nav-links');

if (hamburger) {
  hamburger.addEventListener('click', () => {
    navLinks.classList.toggle('active');
    const icon = hamburger.querySelector('i');
    icon.classList.toggle('fa-bars', !navLinks.classList.contains('active'));
    icon.classList.toggle('fa-times', navLinks.classList.contains('active'));
  });
}

document.querySelectorAll('.nav-links a').forEach(link => {
  link.addEventListener('click', () => {
    if (navLinks.classList.contains('active')) {
      navLinks.classList.remove('active');
      hamburger.querySelector('i').classList.replace('fa-times', 'fa-bars');
    }
  });
});
```

```

// =====
// BOOKING PORTAL (3-Step Flow connected to Python/Flask backend)
// =====

// State
let bookingState = {
  checkIn: '', checkOut: '', guests: 2,
  roomTypeId: null, roomId: null, roomName: '', roomPrice: 0,
  totalNights: 0, totalAmount: 0
};

// — Step helpers —
function goToStep(n) {
  [1, 2, 3].forEach(i => {
    document.getElementById(`booking-step-${i}`).classList.toggle('hidden', i !== n);
    const ind = document.getElementById(`step-ind-${i}`);
    ind.classList.toggle('active', i <= n);
    ind.classList.toggle('completed', i < n);
  });
}

// — Set date minimums —
const checkInInput = document.getElementById('check-in');
const checkOutInput = document.getElementById('check-out');
if (checkInInput) {
  const today = new Date().toISOString().split('T')[0];
  checkInInput.min = today;
  checkOutInput.min = today;
  checkInInput.addEventListener('change', function () {
    checkOutInput.min = this.value;
    if (checkOutInput.value && checkOutInput.value <= this.value) {
      const d = new Date(this.value);
      d.setDate(d.getDate() + 1);
      checkOutInput.value = d.toISOString().split('T')[0];
    }
  });
}

```

6. CSS

```
:root {
  /* Colors */
  --primary-color: #d4af37;
  /* Gold */
  --primary-hover: #c5a028;
  --dark-blue: #0f172a;
  --text-color: #333333;
  --bg-color: #ffffff;
  --bg-light: #f8f9fa;
  --white: #ffffff;

  /* Variables */
  --transition: all 0.3s ease;
  --font-heading: 'Poppins', sans-serif;
  --font-body: 'Roboto', sans-serif;
}

/* Dark Mode Variables */
[data-theme="dark"] {
  --text-color: #e2e8f0;
  --bg-color: #1e293b;
  --bg-light: #0f172a;
  --dark-blue: #0f172a;
  --white: #1e293b;
}

/* Reset & Base Styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}
```

```

/* Buttons */
.btn {
  display: inline-block;
  padding: 12px 28px;
  font-family: var(--font-heading);
  font-weight: 500;
  border-radius: 4px;
  cursor: pointer;
  border: none;
  text-align: center;
  transition: var(--transition);
}

.btn-primary {
  background-color: var(--primary-color);
  color: white #fff;
}

.btn-primary:hover {
  background-color: var(--primary-hover);
  transform: translateY(-2px);
}

.btn-outline {
  background-color: transparent;
  border: 2px solid var(--primary-color);
  color: var(--primary-color);
}

.btn-outline:hover {
  background-color: var(--primary-color);
  color: white #fff;
}

.btn-block {
  display: block;
  width: 100%;
}

```

THANK YOU